Chuleta de la UT5 — Spring Data JPA

La lista **completa** de anotaciones, con sus valores por defecto y sus trampas, está en 3. Referencia de JPA y validaciones.

Dependencias y configuración

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency><groupId>com.h2database</groupId><artifactId>h2</
artifactId>
  <scope>runtime</scope></dependency>
<dependency><groupId>com.mysql</groupId><artifactId>mysql-
connector-j</artifactId>
  <scope>runtime</scope></dependency>
```

dev · H2

```
spring.datasource.url=jdbc:h2:mem:funkos;DB_CLOSE_DELAY=-1
spring.datasource.username=sa
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.h2.console.enabled=true          # /h2-console
spring.jpa.defer-datasource-initialization=true
spring.sql.init.mode=always
spring.jpa.open-in-view=false
```

prod · MySQL

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
spring.h2.console.enabled=false
spring.sql.init.mode=never
spring.jpa.open-in-view=false
```

`ddl-auto`	Qué hace	Dónde
`create-drop`	Borra y crea al arrancar y al parar	**dev / test**
`create`	Borra y crea al arrancar	test
`update`	Añade lo que falta, nunca borra	en ningún sitio serio
`validate`	Comprueba; si no cuadra, **no arranca**	**prod**
`none`	Nada	prod con migraciones

AVISO `create-drop` en producción **borra la base de datos** en cada reinicio. **AVISO** `open-in-view` viene **activado**: apágalo o esconderás los N+1. **AVISO** Sin `defer-datasource-initialization=true`, `data.sql` falla con `Table not found`.

Entidad

```
@Entity
@Table(name = "funkos",
       indexes = @Index(name = "idx_funko_nombre", columnList
= "nombre"),
       uniqueConstraints = @UniqueConstraint(columnNames =
{"titulo", "genero_id"}))
public class Funko {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nombre;

    @Column(nullable = false, precision = 10, scale = 2)   //
dinero
    private BigDecimal precio;

    @Enumerated(EnumType.STRING)                           //
NUNCA ORDINAL
    @Column(length = 20)
    private Categoria categoria;

    @Column(name = "is_deleted", nullable = false)
    private boolean deleted = false;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt = LocalDateTime.now();

    @Version private Long version;                         //
bloqueo optimista

    protected Funko() {}                                   //
JPA lo necesita

    @PreUpdate void alActualizar() { this.updatedAt =
LocalDateTime.now(); }
}
```

AVISO Constructor sin argumentos obligatorio → **una entidad no puede ser** `record`. **AVISO** `@Enumerated` por defecto es `ORDINAL`: guarda la **posición**, y reordenar el `enum` corrompe todos los datos en silencio. **AVISO** Dinero con `BigDecimal` + `precision`/`scale`, nunca `double`.

`@Id`	Cómo genera	Cuándo
`IDENTITY`	Autoincremental de la BD	**MySQL, H2**
`SEQUENCE`	Secuencia con caché	PostgreSQL, inserciones masivas
`AUTO`	Que elija Hibernate	—
`UUID`	En Java, antes de insertar	Ids no enumerables

Relaciones

```
// 1:N — el DUEÑO es el lado MUCHOS (ahí está la FK)
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "categoria_id", nullable = false)
private Categoria categoria;

// y en Categoria, el lado INVERSO
@OneToMany(mappedBy = "categoria", fetch = FetchType.LAZY)
private List<Funko> funkos = new ArrayList<>();

// 1:1
@OneToOne(fetch = FetchType.LAZY, cascade = CascadeType.ALL,
orphanRemoval = true)
@JoinColumn(name = "perfil_id", unique = true)
private Perfil perfil;

// N:M  (solo si la relación NO tiene datos propios)
@ManyToMany(fetch = FetchType.LAZY)
@JoinTable(name = "alumno_modulo",
           joinColumns        = @JoinColumn(name =
"alumno_id"),
           inverseJoinColumns = @JoinColumn(name =
"modulo_id"))
private Set<Modulo> modulos = new HashSet<>();

// Objeto de valor: NO crea tabla
@Embedded private Direccion direccion;      // + @Embeddable
en Direccion
```

	`fetch` por defecto	Lo correcto
`@OneToOne`	`EAGER`	`LAZY`
`@ManyToOne`	`EAGER`	`LAZY`
`@OneToMany`	`LAZY`	`LAZY`
`@ManyToMany`	`LAZY`	`LAZY`

AVISO Los dos `EAGER` por defecto son el problema de rendimiento nº 1 de JPA. Siempre `LAZY`. **AVISO** `@OneToMany` sin `mappedBy` → Hibernate crea una tabla intermedia fantasma. **AVISO** `cascade = ALL` en un `@OneToMany` hace que borrar el padre **borre los hijos**. Casi nunca es lo que quieres: suele ser un 409. **AVISO** En cuanto la relación N:M tiene un **dato propio** (nota, cantidad, fecha), deja de ser `@ManyToMany` y pasa a ser una **entidad intermedia**.

`cascade`	Propaga
`PERSIST` `MERGE` `REMOVE` `REFRESH` `DETACH`	guardar / actualizar / **borrar** / recargar / desvincular
`ALL`	los cinco
`orphanRemoval = true`	sacar un hijo de la colección **lo borra**

Repositorio

```
public interface FunkosRepository
        extends JpaRepository<Funko, Long>,
JpaSpecificationExecutor<Funko> {

    Optional<Funko> findByIdAndDeletedFalse(Long id);
    List<Funko>
findByCategoriaNombreIgnoreCaseAndDeletedFalse(String cat);
    boolean         existsByNombreIgnoreCase(String nombre);
    long            countByCategoriaId(Long id);
    List<Funko>     findTop5ByDeletedFalseOrderByPrecioDesc();
    List<Funko>     findByPrecioBetween(BigDecimal min,
BigDecimal max);
}
```

Ya vienen hechos: `findAll()` · `findAll(Pageable)` · `findById` → `Optional` · `save` · `saveAll` · `deleteById` · `existsById` · `count` · `flush`

Palabra	SQL
`findBy` `countBy` `existsBy` `deleteBy`	`SELECT` `COUNT` `EXISTS` `DELETE`
`And` `Or`	`AND` `OR`
`Between` `LessThan(Equal)` `GreaterThan(Equal)` `After` `Before`	comparaciones
`Like` `Containing` `StartingWith` `EndingWith`	`LIKE`
`IgnoreCase`	`UPPER(…)`
`In` `NotIn` `IsNull` `IsNotNull` `True` `False`	
`OrderBy…Asc/Desc`	`ORDER BY`
`Top3` `First5` `Distinct`	`LIMIT` `DISTINCT`

AVISO Un nombre de campo mal escrito → `PropertyReferenceException` **al ARRANCAR**, no en producción.

`@Query`

```
// JPQL: entidades y campos Java. Se valida al arrancar.
Portable.
@Query("SELECT f FROM Funko f WHERE f.categoria.nombre = :cat
AND f.cantidad > 0")
List<Funko> conStock(@Param("cat") String cat);

// Evitar el N+1 al paginar: JOIN FETCH + countQuery
OBLIGATORIO
@Query(value      = "SELECT f FROM Funko f JOIN FETCH
f.categoria WHERE f.deleted = false",
       countQuery = "SELECT count(f) FROM Funko f WHERE
f.deleted = false")
Page<Funko> findAllConCategoria(Pageable pageable);

// o, más limpio
@EntityGraph(attributePaths = {"categoria"})
Page<Funko> findAll(Pageable pageable);

// Proyección directa a DTO
@Query("SELECT new es.iesx.dto.FunkoResumen(f.nombre,
f.precio) FROM Funko f")
List<FunkoResumen> resumen();

// SQL nativo: tablas y columnas. No portable. No se valida al
arrancar.
@Query(value = "SELECT * FROM funkos WHERE …", nativeQuery =
true)

// Escritura
@Modifying(clearAutomatically = true, flushAutomatically =
true)
@Query("UPDATE Funko f SET f.deleted = true WHERE f.id = :id")
int borradoLogico(@Param("id") Long id);
```

AVISO Nunca concatenes en un `@Query`, ni en el nativo. Usa `:param` o `?1`. **AVISO** `@Modifying` necesita **además** `@Transactional` en quien lo llama. **AVISO** `JOIN FETCH` + `Pageable` **sin** `countQuery` → Hibernate pagina **en memoria** y avisa con `applying in memory`.

Paginación

```
Page<Funko>  findAll(Pageable p);            // + SELECT
count(*): 2 consultas
Slice<Funko> findByDeletedFalse(Pageable p); // 1 consulta,
solo "¿hay más?"
List<Funko>  findByCategoria(String c, Sort s);
```

```
@GetMapping
public ResponseEntity<Page<FunkoResponse>> findAll(
        @PageableDefault(size = 20, sort = "id") Pageable
pageable) {
    return
ResponseEntity.ok(servicio.findAll(sanear(pageable)));
}

private Pageable sanear(Pageable p) {                    //
lista BLANCA + tope
    var ok = Set.of("id", "nombre", "precio", "createdAt");
    var orden = p.getSort().stream()
                 .filter(o ->
ok.contains(o.getProperty())).toList();
    return PageRequest.of(p.getPageNumber(),
Math.min(p.getPageSize(), 100),
                          orden.isEmpty() ? Sort.by("id") :
Sort.by(orden));
}
```

```
spring.data.web.pageable.default-page-size=20
spring.data.web.pageable.max-page-size=100
```

`?page=0&size=20&sort=precio,desc&sort=nombre,asc` · `page` **empieza en 0**

`Page`: `getContent()` `getTotalElements()` `getTotalPages()` `getNumber()` `hasNext()` `isLast()` `map(fn)`

```
return repositorio.findAll(spec,
pageable).map(mapper::toResponse);   // Page<DTO>
```

AVISO Sin tope de `size`, `?size=1000000` carga la tabla entera: has paginado para nada. **AVISO** Sin lista blanca de `sort`, se puede ordenar por **cualquier** campo de la entidad.

`Specification` — criterios combinables

```
public interface R extends JpaRepository<Funko,Long>,
JpaSpecificationExecutor<Funko> {}
```

```
Specification<Funko> spec = (root, q, cb) ->
cb.isFalse(root.get("deleted"));

spec = categoria.map(c -> spec.and((root, q, cb) ->

cb.equal(cb.upper(root.get("categoria").get("nombre")),
c.toUpperCase())))
        .orElse(spec);

spec = precioMin.map(p -> spec.and((root, q, cb) ->
        cb.greaterThanOrEqualTo(root.get("precio"),
p))).orElse(spec);

spec = nombre.map(n -> spec.and((root, q, cb) ->
        cb.like(cb.lower(root.get("nombre")), "%" +
n.toLowerCase() + "%")))
        .orElse(spec);

return repositorio.findAll(spec,
pageable).map(mapper::toResponse);
```

`cb.equal` `notEqual` `like` `greaterThan(OrEqualTo)` `lessThan(OrEqualTo)` `between` `isNull` `isTrue` `isFalse` `in` `and` `or` `not` · `cb.upper` `cb.lower`

AVISO Con 4 filtros opcionales hay **16 combinaciones**. Con `if` anidados es inmantenible.

`@Transactional`

```
@Transactional(readOnly = true)             // lecturas
@Transactional                              // escrituras
@Transactional(rollbackFor = Exception.class)
```

AVISO **Solo revierte con `RuntimeException`.** Una comprobada **confirma** la transacción a medias. **AVISO** Funciona **por proxy**: `this.save(...)` no abre transacción. Igual que `@Cacheable`. **AVISO** Si **capturas** la excepción dentro, **no hay rollback**: tiene que salir del método. **AVISO** Va en el **servicio**, no en el repositorio: dos llamadas a `JpaRepository` son **dos** transacciones.

```
@Version private Long version;    // →
ObjectOptimisticLockingFailureException → 409
```

Dentro de una transacción las entidades están **gestionadas**: un `setter` ya persiste, sin `save()`.

Tests

```
// 1 · Repositorio — solo JPA, H2, rollback automático. ~1 s
@DataJpaTest @ActiveProfiles("test")
class RepoTest {
    @Autowired FunkosRepository repo;
    @Autowired TestEntityManager em;

    @Test void noDevuelveBorrados() {
        em.persistAndFlush(funko("Goku", false));
        em.persistAndFlush(funko("Naruto", true));
        assertThat(repo.findByDeletedFalse()).hasSize(1);
    }
}
```

```
// 2 · Servicio — sin Spring. ms
@ExtendWith(MockitoExtension.class)
class ServicioTest {
    @Mock FunkosRepository repo;  @Spy FunkoMapper mapper =
new FunkoMapper();
    @InjectMocks FunkosServiceImpl servicio;

    @Test void noBorraConStock() {

when(repo.findByIdAndDeletedFalse(1L)).thenReturn(Optional.of(fu
        assertThrows(FunkoConflictException.class, () ->
servicio.deleteById(1L));
        verify(repo, never()).borradoLogico(anyLong());
    }
}
```

```
// 3 · Integración — todo junto. ~3 s
@SpringBootTest @AutoConfigureMockMvc @ActiveProfiles("test")
class IntegracionTest {
    @Autowired MockMvc mockMvc;

    @Test void paginado() throws Exception {
        mockMvc.perform(get("/api/v1/funkos?page=0&size=2"))
               .andExpect(status().isOk())
               .andExpect(jsonPath("$.content", hasSize(2)))
               .andExpect(jsonPath("$.totalElements").exists());
    }
}
```

AVISO Sin `persistAndFlush`, el `INSERT` se queda en la caché de Hibernate y la consulta no lo ve.

```
./mvnw test
./mvnw test -Dtest="*RepositoryTest"
./mvnw test -Dtest="FunkosRepositoryTest#noDevuelveBorrados"
```

Docker

```
services:
  api:
    build: .
    ports: ["8080:8080"]
    env_file: .env
    environment:
      SPRING_PROFILES_ACTIVE: prod
      DB_URL: jdbc:mysql://db:3306/funkos      # ← "db", NO
localhost
    depends_on:
      db: { condition: service_healthy }       # ← sin esto,
falla al arrancar
  db:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: funkos
    volumes: [dbdata:/var/lib/mysql]
    healthcheck:
      test: ["CMD","mysqladmin","ping","-h","localhost"]
      interval: 5s
      retries: 10
volumes: { dbdata: }
```

Los diez errores del examen

	Síntoma	Causa
1	`LazyInitializationException`	Se devuelve la entidad en vez del DTO
2	100 funkos → 101 consultas	`@ManyToOne` en `EAGER`, o falta `JOIN FETCH`
3	`StackOverflowError` al serializar	Relación bidireccional devuelta tal cual
4	Tabla intermedia que no pediste	`@OneToMany` sin `mappedBy`
5	Borrar un género borra sus películas	`cascade = CascadeType.ALL`
6	Los datos de Disney pasan a ser de Marvel	`@Enumerated` sin `EnumType.STRING`
7	`Table "FUNKOS" not found` al arrancar	Falta `defer-datasource-initialization=true`
8	`TransactionRequiredException`	`@Modifying` sin `@Transactional`
9	La transacción se confirma tras un fallo	Excepción comprobada, o capturada dentro
10	Base de datos vacía tras reiniciar	`ddl-auto=create-drop` en `prod`